Java Data types and Variables

Primitive Data Types

	Type	Size (in Bits)
• Integer		
	byte	8
	short	16
	int	32
	long	64
• Float	float	32
	double	64
• Character		
• Boolean	1/0	

Types and Variables

- Every value has a type
- Variable declaration examples:
  String greeting = "Hello, World!";
  PrintStream printer = System.out;
  int Number = 13;
  float num = 21.0;
- Variables
  - Store values
  - Can be used in place of the objects they store

Variable Definition

- *typeName variableName = value*;
  or
  typeName variableName;

Example:

- String greeting = "Hello, Dave!";

Purpose:

- To define a new variable of a particular type and optionally supply an initial value

Identifiers

- Identifier: name of a variable, method, or class
- Rules for identifiers in Java:
  - Can be made up of letters, digits, and the underscore (_) character
  - Cannot start with a digit
  - Cannot use other symbols such as ? or %
  - Spaces are not permitted inside identifiers
  - You cannot use reserved words
  - They are case sensitive

- By convention, variable names start with a lowercase letter
- By convention, class names start with an uppercase letter

Self Check

- What is the type of the values 0 and "0"?

  int and String

- Which of the following are legal identifiers?

  Greeting1
  g
  void
  101dalmatians
  Hello, World
  <greeting>

  Only the first two are legal identifiers

- Define a variable to hold your name

  String myName = "John Q. Public";

The Assignment Operator

- Assignment operator: =
- Not used as a statement about equality

- Used to change the value of a variable
  int luckyNumber = 13;
  luckyNumber = 12;

Uninitialized Variables

- int luckyNumber;
  System.out.println(luckyNumber); ????
  - // ERROR - uninitialized variable

Syntax : Assignment

- *variableName* = *value*;

Self Check

- Is 12 = 12 a valid expression in the Java language?
- How do you change the value of the greeting variable to "Hello"?

Answers

- No, the left-hand side of the = operator must be a variable
- greeting = "Hello";
  Note that
  String greeting = "Hello";
  is not the right answer–that statement defines a new variable

String Methods

- **length**: counts the number of characters in a string

```
String greeting = "Hello, World!";
int n = greeting.length();          // sets n to 13
```

- **toUpperCase**: creates another String object that contains the characters of the original string, with lowercase letters converted to uppercase

```
String river = "Mississippi";          // sets bigRiver to "MISSISSIPPI"
String  bigRiver = river.toUpperCase();
```

- When applying a method to an object, make sure method is defined in the appropriate class

```
System.out.length();
                                    // This method call is an error
```

```
greeting = "Hello, World!";
river = "Mississippi";
```

Self Check

- How can you compute the length of the string "Mississippi"?
- How can you print out the uppercase version of "Hello, World!"?
- Is it legal to call river.println()? Why or why not?

Answers

- river.length() or "Mississippi".length()
- System.out.println(greeting.toUpperCase());
- It is not legal. The variable river has type String. The println method is not a method of the String class.

Implicit and Explicit Parameters

- Parameter (explicit parameter):
  - Input to a method.
    - System.out.println(greeting)
  - Not all methods have explicit parameters.
    - greeting.length() // has no explicit parameter

- Implicit parameter: The object on which a method is invoked

  greeting.length()

Return Values

- Return value: A result that the method has computed, for use

```
int n = greeting.length();
                        // return value stored in n
```

Passing Return Values

- You can also use the return value as a parameter of another method:
  System.out.println(greeting.length());

- Not all methods return values
  - Example: println

A More Complex Call

- Let, String river=“Mississippi”;
- replace method carries out a search-and-replace operation

```
river.replace("issipp", "our")
                    // constructs a new string ("Missouri")
```

- This method call has
  - one implicit parameter: the string "Mississippi"
  - two explicit parameters: the strings "issipp" and "our"
  - a return value: the string "Missouri"

Method Definitions

- Method definition specifies types of explicit parameters and return value
- Type of implicit parameter (is object); not mentioned in method definition
- Example: Class String defines,

```
public int length()
        // return type: int
        // no explicit parameter

public String replace(String target, String replacement)
        // return type: String;
        // two explicit parameters of type String
```

Method Definitions

- If method returns no value, the return type is declared as void

  public void println(String output) // in class PrintStream

- A method name is overloaded if a class has more than one method with the same name (but different parameter types)

  public void println(String output)
  public void println(int output)

river=“Mississippi”;

greeting = "Hello, World!";

Self Check

- What are the implicit parameters, explicit parameters, and return values in the method call river.length()?
- What is the result of the call river.replace("p", "s")?
- What is the result of the call greeting.replace("World", "Dave").length()?
- How is the toUpperCase method defined in the String class?

- **Answers**
- The implicit parameter is river. There is no explicit parameter. The return value is 11
- "Missississi"
- 12
- As public String toUpperCase(), with no explicit parameter and return type String.

Number Types

- Integers: short, int, long
  13
- Floating point numbers: float, double
  1.3
  0.00013
- When a floating-point number is multiplied or divided by 10, only the position of the decimal point changes; it "floats". This representation is related to the "scientific" notation 1.3×10^{-4}.
  1.3E-4 // 1.3×10^{-4} written in Java
- Numbers are not objects; numbers types are primitive types

Arithmetic Operations

- Operators: + - *
  10 + n
  n - 1
  10 * n // 10 × n

- As in mathematics, the * operator binds more strongly than the + operator
  x + y * 2 // means the sum of x and y * 2
  (x + y) * 2 // multiplies the sum of x and y with 2

Self Check

- Which number type would you use for storing the area of a circle?
- Is the expression 13.println() ok?
- Write an expression to compute the average of the values x and y.

Answers

- double
- An int is not an object, and you cannot call a method on it
- (x + y) * 0.5

Rectangular Shapes and Rectangle Objects

- Objects of type Rectangle *describe* rectangular shapes

- A Rectangle object isn't a rectangular shape– it is an object that contains a set of numbers that describe the rectangle

Constructing Objects

- new Rectangle(5, 10, 20, 30)

  Detail:
  - The new operator makes a Rectangle object
  - It uses the parameters (in this case, 5, 10, 20, and 30) to initialize the data of the object
  - It returns the object
- Usually the output of the new operator is stored in a variable

```
Rectangle box = new Rectangle(5, 10, 20, 30);
```

Constructing Objects

- The process of creating a new object is called *construction*
- The four values 5, 10, 20, and 30 are called the *construction parameters*

Self Check

- How do you construct a square with center (100, 100) and side length 20?

- What does the following statement print?
  System.out.println(new Rectangle().getWidth());

Answers

- new Rectangle(90, 90, 20, 20)
- 0.0